Computer Architecture Project

Undergraduate Thesis Report

Submitted in partial fulfillment of the requirements of
CS F342 Computer Architecture

By

Apoorv Vuppulury
ID No. 2022AAPS0504H

Under the supervision of:

S. Gurunarayanan
Senior Professor
BITS Pilani, Hyderabad Campus

,
September 2025

Project
Set - 17

Statement: Design a 5-stage (as discussed in the lecture) pipeline RISC processor (with hazard detection and data forwarding unit as necessary) that can execute the following instructions:

0000 AND reg1, reg2, reg3
0004 ORI reg4, reg1, ABCD
0008 SW reg4, 8(reg5)
0012 NAND reg7, reg4, reg6
0016 ADD reg9, reg7, reg8

Initialize the register file with the following data
reg2 = 90567 reg3 = 9778A
reg5 = 4 reg6 = ACEE1
reg8 = 18765

The instruction format is as follows:

31 28	27 24	23 20	19 16	15 0
OPCODE	Dest Reg (WN)	Source Reg 1 (RN1)	Source Reg 2 (RN2)	Immediate value

Opcode for each instruction:
Instruction 1: 0000
Instruction 2: 0001
Instruction 3: 0011
Instruction 4: 0111
Instruction 5: 1111

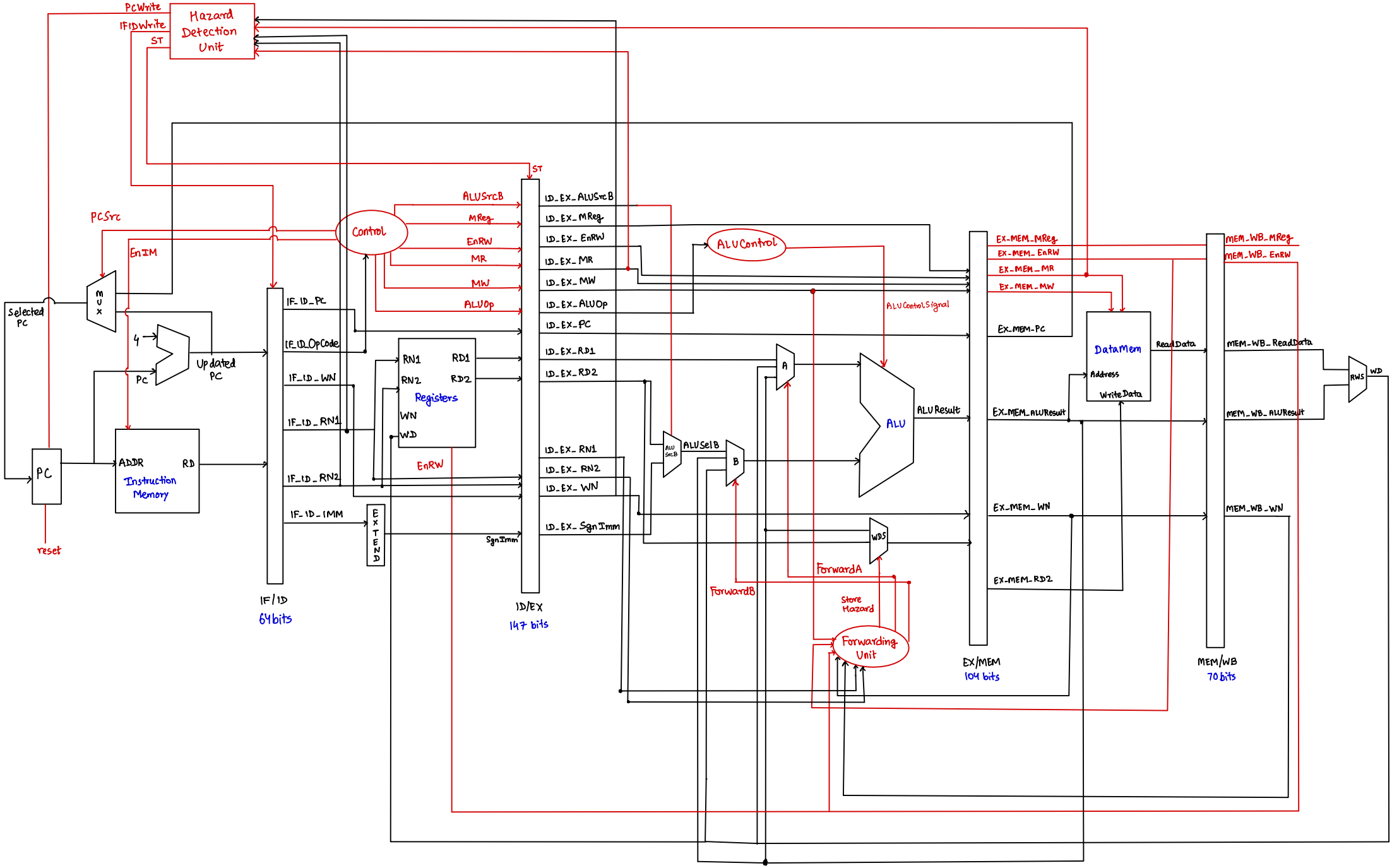
The processor has the following control signals:

- ALUSrc - Select the second input of ALU
- ALUOp (2 bits) - Control ALU operation
- MR - Read data from memory
- MW - Write data into memory
- MReg - Move data from memory to register
- EnIM - Read instruction memory contents
- EnRW - Write data into the register file
- FA - Forward A mux control (used in data forwarding circuitry)
- FB - Forward B mux control (used in data forwarding circuitry)
- IFIDWrite - Disable IF/ID change (used in hazard detection circuit)

- PCWrite - Disable PC change (used in hazard detection circuit)
- ST - Control signal of mux which changes all control signals to zero (used in hazard detection circuit)

Initialize PC with all zeros. The instruction memory is of size 32 bytes, and the processor has 16 registers, numbered from reg0 to reg15, and the registers are 32 bits wide. A read operation from the instruction memory outputs 4 consecutive bytes of information (starting from the byte address provided to the memory) at the positive edge of the clock if the EnIM control signal is high. Assume that the register file has two 32-bit read ports: RD1 and RD2, for data reading and one 32-bit write port: WD, for data writing. At the rising edge of the clock, the read ports RD1 and RD2 output the data from the registers whose addresses are available at RN1 and RN2, respectively. At the falling edge of a clock, data is written via the write port WD to the register file whose address is present at WN if the EnRW signal is true. Design the data memory size as per the requirement. All the data are in hexadecimal format unless specified.

Draw the detailed architecture-level diagram of the processor, depicting all the blocks (e.g., register file, instruction memory, ALU, PC, combinational functional blocks, hazard detection unit, forwarding unit, etc.). Describe each block individually, and create behavioral verilog models for each architectural block separately. Build the top-level structural model of the processor by instantiating and interconnecting the individual architectural blocks. Specify the size and format of all pipeline registers including the fields holding the decoded control signals as well as data. Show all the input, output, and control signal waveforms in the report.


PCWrite
IF/IDWrite
ST
Hazard Detection Unit
PCSrc
EnIM
Control
ALUSrcB
MReg
EnRW
MR
MW
ALUOp
ST
ID_EX_ALUSrcB
ID_EX_MReg
ID_EX_EnRW
ID_EX_MR
ID_EX_MW
ID_EX_ALUOp
ID_EX_PC
ID_EX_RD1
ID_EX_RD2
ID_EX_RN1
ID_EX_RN2
ID_EX_WN
ID_EX_SgnImm
ALUControl
ALU Control Signal
Selected PC
MUX
4
PC
Updated PC
PC
ADDR
RD
Instruction Memory
reset
IF_ID_PC
IF_ID_OpCode
IF_ID_WN
IF_ID_RN1
IF_ID_RN2
IF_ID_IMM
EXTEND
RN1
RN2
WN
WD
RD1
RD2
Registers
EnRW
IF/ID
64 bits
ALU Src B
ALUSelB
A
B
ALU
ALU Result
WDS
ForwardA
ForwardB
Store Hazard
Forwarding Unit
EX_MEM_MReg
EX_MEM_EnRW
EX_MEM_MR
EX_MEM_MW
EX_MEM_PC
EX_MEM_ALUResult
EX_MEM_WN
EX_MEM_RD2
EX/MEM
104 bits
DataMem
Address
WriteData
ReadData
MEM_WB_MReg
MEM_WB_EnRW
MEM_WB_ReadData
MEM_WB_ALUResult
MEM_WB_WN
RWS
WD
MEM/WB
70 bits
ID/EX
147 bits

Program counter:

```
`timescale 1ns / 1ps

module ProgramCounter(
input Reset,PCWrite, Clk,
input [31:0] SelectedPC,
output reg [31:0] PC
);

always@(posedge Clk)
begin
    if(Reset)
        PC<=32'd0;
    else if(PCWrite)
        PC<=PC;
    else
        PC<=SelectedPC;
end
endmodule
```

Adder:

```
`timescale 1ns / 1ps

module Adder(
input [31:0]PC,
output [31:0] UpdatedPC
);

assign UpdatedPC = PC+4;

endmodule
```

MUXPCSrc:

```
`timescale 1ns / 1ps

module MUXPCSrc(
input PCSrc,
input [31:0] UpdatedPC, EX_MEM_PC,
output [31:0]SelectedPC);

assign SelectedPC= PCSrc ? EX_MEM_PC : UpdatedPC;

endmodule
```

InstructionMemory:

```
`timescale 1ns / 1ps


module InstructionMemory(
input EnIM,
input [31:0]ADDR,
output reg [31:0] RD
);

reg [7:0]InstMem [0:31];

always@(*)
begin
    if(EnIM)
        RD ={InstMem[ADDR],InstMem[ADDR+1],InstMem[ADDR+2],InstMem[ADDR+3]};
end

endmodule
```

IF_ID:

```
`timescale 1ns / 1ps

module IF_ID(input Clk,
input IFIDWrite,
input [31:0] Instruction, UpdatedPC,
output reg [3:0] IF_ID_OpCode, IF_ID_WN, IF_ID_RN1, IF_ID_RN2,
output reg [15:0] IF_ID_IMM,
output reg [31:0] IF_ID_PC
);

always@(posedge Clk)
begin
    if(IFIDWrite)
    begin
        IF_ID_OpCode<=IF_ID_OpCode;
        IF_ID_WN<=IF_ID_WN;
        IF_ID_RN1<=IF_ID_RN1;
        IF_ID_RN2<=IF_ID_RN2;
        IF_ID_IMM<=IF_ID_IMM;
        IF_ID_PC<=IF_ID_PC;
    end
    else
    begin
        IF_ID_OpCode<=Instruction[31:28];
        IF_ID_WN<=Instruction[27:24];
        IF_ID_RN1<=Instruction[23:20];
        IF_ID_RN2<=Instruction[19:16];
        IF_ID_IMM<=Instruction[15:0];
        IF_ID_PC<=UpdatedPC;
    end
end
endmodule
```

RegisterFile:

```
`timescale 1ns / 1ps

module RegisterFile(
input Clk, EnRW,
input [3:0] WN, RN1, RN2,
input [31:0] WD,
output[31:0] RD1,RD2
);

reg [31:0] Registers [0:15];

always@(negedge Clk)
begin
    if(EnRW)
        Registers[WN]<=WD;
end

assign RD1= Registers[RN1];
assign RD2= Registers[RN2];

endmodule
```

MainControl:

```
`timescale 1ns / 1ps

module MainControl(input Clk,
input [3:0] OpCode,
output PCSrc, EnIM,
output reg ALUSrcB, MReg, EnRW, MR, MW,
output reg [1:0] ALUOp
);

assign PCSrc=0; //no branching operation
assign EnIM=1;
always@(negedge Clk)
begin
    if(OpCode == 4'b0000) //AND
    begin
        ALUSrcB<=1;
        MReg<=0;
        EnRW<=1;
        MR<=0;
        MW<=0;
        ALUOp <=2'b00;
    end

    else if(OpCode == 4'b0001) //ORI
    begin
        ALUSrcB<=0;
        MReg<=0;
        EnRW<=1;
        MR<=0;
        MW<=0;
        ALUOp <=2'b01;
    end
    else if(OpCode == 4'b0011) //SW
    begin
        ALUSrcB<=0;
        MReg<=1;
        EnRW<=0;
        MR<=0;
        MW<=1;
        ALUOp <=2'b10;
    end
```

```
42
43      else if(OpCode == 4'b0111) //NAND
44      begin
45          ALUSrcB<=1;
46          MReg<=0;
47          EnRW<=1;
48          MR<=0;
49          MW<=0;
50          ALUOp <=2'b11;
51      end
52
53      else if(OpCode == 4'b1111) //ADD
54      begin
55          ALUSrcB<=1;
56          MReg<=0;
57          EnRW<=1;
58          MR<=0;
59          MW<=0;
60          ALUOp <=2'b10;
61      end
62  end
63
64  endmodule
```

Extend:

```
1   `timescale 1ns / 1ps
2
3   module Extend(
4   input [15:0]IMM,
5   output [31:0]SgnIMM
6   );
7
8   assign SgnIMM = {{16{IMM[15]}}, IMM};
9
10  endmodule
```

ID_EX:

```
1   `timescale 1ns / 1ps
2
3   module ID_EX(input Clk, ST, ALUSrcB, MReg, EnRW, MR, MW,
4   input [1:0] ALUOp,
5   input [3:0] WN, RN1, RN2,
6   input [31:0] RD1, RD2, IF_ID_PC, SgnIMM,
7   output reg ID_EX_PCSrc, ID_EX_ALUSrcB, ID_EX_MReg, ID_EX_EnRW, ID_EX_MR, ID_EX_MW,
8   output reg [1:0] ID_EX_ALUOp,
9   output reg [3:0] ID_EX_WN, ID_EX_RN1, ID_EX_RN2,
10  output reg [31:0] ID_EX_RD1, ID_EX_RD2, ID_EX_PC, ID_EX_SgnIMM
11  );
12
```

```
always@(posedge Clk)
begin
    if(ST)
    begin
    ID_EX_ALUSrcB<=1'b0;
    ID_EX_MReg<=1'b0;
    ID_EX_EnRW<=1'b0;
    ID_EX_MR<=1'b0;
    ID_EX_MW<=1'b0;
    ID_EX_ALUOp<=2'b0;
    ID_EX_WN<=4'b0;
    ID_EX_RD1<=32'b0;
    ID_EX_RD2<=32'b0;
    ID_EX_PC<=32'b0;
    ID_EX_SgnIMM<=32'b0;
    ID_EX_RN1<=4'b0;
    ID_EX_RN2<=4'b0;
    end

    //ID_EX_PCSrc<=PCSrc;
    else
    begin
    ID_EX_ALUSrcB<=ALUSrcB;
    ID_EX_MReg<=MReg;
    ID_EX_EnRW<=EnRW;
    ID_EX_MR<=MR;
    ID_EX_MW<=MW;
    ID_EX_ALUOp<=ALUOp;
    ID_EX_WN<=WN;
    ID_EX_RD1<=RD1;
    ID_EX_RD2<=RD2;
    ID_EX_PC<=IF_ID_PC;
    ID_EX_SgnIMM<=SgnIMM;
    ID_EX_RN1<=RN1;
    ID_EX_RN2<=RN2;
    end
end

endmodule
```

ALUControl:

```
`timescale 1ns / 1ps

module ALUControl(
input [1:0] ID_EX_ALUOp,
output reg [1:0] ALUControlSignal
);

always @(*) begin
    case (ID_EX_ALUOp)
        2'b00: ALUControlSignal = 2'b00; // AND
        2'b01: ALUControlSignal = 2'b01; // ORI
        2'b10: ALUControlSignal = 2'b10; // ADD
        2'b11: ALUControlSignal = 2'b11; // NAND

        default: ALUControlSignal = 2'b10;
    endcase
end
endmodule
```

ALU:

```
`timescale 1ns / 1ps
module ALU (
input [31:0] ALUInA, ALUInB,
input [1:0] ALUControlSignal,
output reg [31:0] ALUResult
);

wire [31:0] Sum, Diff;

always @(*) begin
    case (ALUControlSignal)
        2'b00: ALUResult = (ALUInA & ALUInB);  //AND operation
        2'b01: ALUResult = (ALUInA | ALUInB);  //OR operation
        2'b10: ALUResult = ALUInA + ALUInB;  //ADD operation
        2'b11: ALUResult = ~(ALUInA & ALUInB);  //NAND operation
        default: ALUResult = 32'b0;
    endcase
end

endmodule
```

MUXALUSrcB:

```
`timescale 1ns / 1ps


module MUXALUSrcB(
input ALUSrcB,
input [31:0] ID_EX_RD2, ID_EX_SgnIMM,
output [31:0] ALUSelB
);

assign ALUSelB = ALUSrcB ? ID_EX_RD2 : ID_EX_SgnIMM;

endmodule
```

MUXForwardA:

```
`timescale 1ns / 1ps

module MUXForwardA(
input [1:0] ForwardA,
input [31:0] EX_MEM_ALUResult, WD, ID_EX_RD1,
output reg [31:0] ALUInA
);

always@(*)
begin
    case(ForwardA)
        2'b00 : ALUInA <= ID_EX_RD1;
        2'b01 : ALUInA <= WD;
        2'b10 : ALUInA <= EX_MEM_ALUResult;
        default : ALUInA <=32'b0;
    endcase
end
endmodule
```

MUXForwardB:

```
`timescale 1ns / 1ps

module MUXForwardB(
input [1:0]ForwardB,
input [31:0] EX_MEM_ALUResult, WD, ALUSelB,
output reg [31:0] ALUInB
);

always@(*)
begin
    case(ForwardB)
        2'b00 : ALUInB <= ALUSelB;
        2'b01 : ALUInB <= WD;
        2'b10 : ALUInB <= EX_MEM_ALUResult;
        default : ALUInB <=32'b0;
    endcase
end
endmodule
```

MUXWriteDataSrc:

```
`timescale 1ns / 1ps

module MUXWriteDataSrc(
input StoreHazard,
input [31:0] ID_EX_RD2,
input [31:0] EX_MEM_ALUResult,
output [31:0] WriteData
);

begin
    assign WriteData = StoreHazard ? EX_MEM_ALUResult : ID_EX_RD2;
end
endmodule
```

EX_MEM:

```
`timescale 1ns / 1ps

module EX_MEM(
input Clk,
input ID_EX_MReg, ID_EX_EnRW, ID_EX_MW, ID_EX_MR,
input [3:0] ID_EX_WN,
input [31:0] ALUResult, ID_EX_PC, ID_EX_RD2,
output reg  EX_MEM_MReg, EX_MEM_EnRW, EX_MEM_MW, EX_MEM_MR,
output reg [3:0] EX_MEM_WN,
output reg [31:0] EX_MEM_ALUResult, EX_MEM_PC, EX_MEM_RD2
);

always@(posedge Clk)
begin
    EX_MEM_MReg<=ID_EX_MReg;
    EX_MEM_EnRW<=ID_EX_EnRW;
    EX_MEM_MW<=ID_EX_MW;
    EX_MEM_MR<=ID_EX_MR;
    EX_MEM_WN<=ID_EX_WN;
    EX_MEM_ALUResult<=ALUResult;
    EX_MEM_PC<=ID_EX_PC;
    EX_MEM_RD2<=ID_EX_RD2;
end
endmodule
```

DataMemory:

```
`timescale 1ns / 1ps

module DataMemory(
input [31:0] Address, WriteData,
input MemRead, MemWrite,
output reg [31:0]ReadData
);

reg [7:0] DataMem [0:15];

always@(*)
begin
if(MemRead)
    ReadData = {DataMem[Address],DataMem[Address+1],DataMem[Address+2],DataMem[Address+3]};
if(MemWrite)
    {DataMem[Address],DataMem[Address+1],DataMem[Address+2],DataMem[Address+3]}<=WriteData;
end

endmodule
```

MEM_WB:

```
`timescale 1ns / 1ps

module MEM_WB(
input Clk,
input EX_MEM_MReg, EX_MEM_EnRW,
input [3:0] EX_MEM_WN,
input [31:0] ReadData, EX_MEM_ALUResult,
output reg MEM_WB_MReg, MEM_WB_EnRW,
output reg [3:0] MEM_WB_WN,
output reg [31:0] MEM_WB_ALUResult, MEM_WB_ReadData
);

always@(posedge Clk)
begin
    MEM_WB_MReg<=EX_MEM_MReg;
    MEM_WB_EnRW<=EX_MEM_EnRW;
    MEM_WB_ReadData<=ReadData;
    MEM_WB_ALUResult<=EX_MEM_ALUResult;
    MEM_WB_WN<=EX_MEM_WN;
end
endmodule
```

MUXRegWriteSrc:

```
`timescale 1ns / 1ps

module MUXRegWriteSrc(
input MEM_WB_MReg,
input [31:0] MEM_WB_ALUResult, MEM_WB_ReadData,
output [31:0] WD
);

assign WD= MEM_WB_MReg ? MEM_WB_ReadData : MEM_WB_ALUResult;

endmodule
```

ForwardingUnit:

```
`timescale 1ns / 1ps

module ForwardingUnit(
input ID_EX_MW, EX_MEM_EnRW, MEM_WB_EnRW,
input [3:0] EX_MEM_WN, MEM_WB_WN, ID_EX_RN1, ID_EX_RN2,
output reg StoreHazard,
output reg [1:0] ForwardA, ForwardB
);

always@(*)
begin
    if( EX_MEM_EnRW & (ID_EX_RN1 == EX_MEM_WN) & (EX_MEM_WN != 4'b0))
        ForwardA <= 2'b10;
    else if(MEM_WB_EnRW & (ID_EX_RN1 == MEM_WB_WN) & (MEM_WB_WN != 4'b0))
        ForwardA <= 2'b01;
    else
        ForwardA <= 2'b00;

    if(ID_EX_MW & (ID_EX_RN2 == EX_MEM_WN))
        begin
        ForwardB<=2'b00;
        StoreHazard<=1;
        end
    else if(EX_MEM_EnRW & (ID_EX_RN2 == EX_MEM_WN) & ((EX_MEM_WN != 4'b0)))
        begin
        ForwardB <= 2'b10;
        StoreHazard<=0;
        end
    else if(MEM_WB_EnRW & (ID_EX_RN2 == MEM_WB_WN) & (MEM_WB_WN != 4'b0))
        begin
        ForwardB <= 2'b01;
        StoreHazard<=0;
        end
    else
        begin
        ForwardB <= 2'b00;
        StoreHazard<=0;
        end
end
endmodule
```

Stalling:

```
`timescale 1ns / 1ps

module Stalling(
input ID_EX_MR,
input [3:0] ID_EX_WN, IF_ID_RN1, IF_ID_RN2,
output reg IFIDWrite, PCWrite, ST
);

always@(*)
begin
if(ID_EX_MR &(ID_EX_WN==IF_ID_RN1 | ID_EX_WN==IF_ID_RN2))
    begin
        ST<=1;
        IFIDWrite<=1;
        PCWrite<=1;
    end
else
    ST<=0;
    IFIDWrite<=0;
    PCWrite<=0;
end

endmodule
```

ProcessorPipeline:

```
`timescale 1ns / 1ps

module ProcessorPipeline(
input Clk,Reset
);

wire StoreHazard, ST, IFIDWrite, PCWrite, PCSrc, EnIM, EnRW, MEM_WB_EnRW, ALUSrcB, MReg, MR, MW;
wire ID_EX_ALUSrcB, ID_EX_MReg, ID_EX_EnRW, ID_EX_MR, ID_EX_MW, EX_MEM_MReg, EX_MEM_EnRW, EX_MEM_MW, EX_MEM_MR, MEM_WB_MReg, ID_EX_Write;
wire [1:0] ALUOp, ID_EX_ALUOp, ALUControlSignal, ForwardA, ForwardB;
wire [3:0] IF_ID_OpCode, IF_ID_WN, IF_ID_RN1, IF_ID_RN2, MEM_WB_WN, WN, ID_EX_WN, EX_MEM_WN, ID_EX_RN1, ID_EX_RN2;
wire [15:0] IF_ID_IMM;
wire [31:0] PC,UpdatedPC,SelectedPC,Instruction,IF_ID_PC,WD,RD1,RD2,SgnIMM,ID_EX_RD1,ID_EX_RD2, ID_EX_PC,ID_EX_SgnIMM;
wire [31:0] ALUSelB,ALUResult,EX_MEM_ALUResult,EX_MEM_PC,EX_MEM_RD2,ReadData,MEM_WB_PC,MEM_WB_ALUResult,MEM_WB_ReadData,ALUInA,ALUInB,WriteData;


ProgramCounter pc(
    .Reset(Reset),
    .PCWrite(PCWrite),
    .Clk(Clk),
    .SelectedPC(SelectedPC),
    .PC(PC)
);

Adder adder(
    .PC(PC),
    .UpdatedPC(UpdatedPC)
);

MUXPCSrc muxpcsrc(
    .PCSrc(PCSrc),
    .UpdatedPC(UpdatedPC),
    .EX_MEM_PC(EX_MEM_PC),
    .SelectedPC(SelectedPC)
);

InstructionMemory im(
    .EnIM(EnIM),
    .ADDR(PC),
    .RD(Instruction)
);

IF_ID ifid(
    .Clk(Clk),
    .IFIDWrite(IFIDWrite),
    .Instruction(Instruction),
    .UpdatedPC(UpdatedPC),
    .IF_ID_OpCode(IF_ID_OpCode),
    .IF_ID_WN(IF_ID_WN),
    .IF_ID_RN1(IF_ID_RN1),
    .IF_ID_RN2(IF_ID_RN2),
    .IF_ID_IMM(IF_ID_IMM),
    .IF_ID_PC(IF_ID_PC)
);
```

```
RegisterFile regfile(
    .Clk(Clk),
    .EnRW(MEM_WB_EnRW),
    .WN(MEM_WB_WN),
    .RN1(IF_ID_RN1),
    .RN2(IF_ID_RN2),
    .WD(WD),
    .RD1(RD1),
    .RD2(RD2)
);

MainControl mc(
    .Clk(Clk),
    .OpCode(IF_ID_OpCode),
    .PCSrc(PCSrc),
    .EnIM(EnIM),
    .ALUSrcB(ALUSrcB),
    .MReg(MReg),
    .EnRW(EnRW),
    .MR(MR),
    .MW(MW),
    .ALUOp(ALUOp)
);

Extend ex(
    .IMM(IF_ID_IMM),
    .SgnIMM(SgnIMM)
);


ID_EX idex(
    .Clk(Clk),
    .ST(ST),
    .ALUSrcB(ALUSrcB),
    .MReg(MReg),
    .EnRW(EnRW),
    .MR(MR),
    .MW(MW),
    .RN1(IF_ID_RN1),
    .RN2(IF_ID_RN2),
    .WN(IF_ID_WN),
    .RD1(RD1),
    .RD2(RD2),
    .IF_ID_PC(IF_ID_PC),
    .SgnIMM(SgnIMM),
    .ALUOp(ALUOp),
    .ID_EX_ALUSrcB(ID_EX_ALUSrcB),
    .ID_EX_MReg(ID_EX_MReg),
    .ID_EX_EnRW(ID_EX_EnRW),
    .ID_EX_MR(ID_EX_MR),
    .ID_EX_MW(ID_EX_MW),
    .ID_EX_ALUOp(ID_EX_ALUOp),
    .ID_EX_RN1(ID_EX_RN1),
    .ID_EX_RN2(ID_EX_RN2),
    .ID_EX_WN(ID_EX_WN),
    .ID_EX_RD1(ID_EX_RD1),
    .ID_EX_RD2(ID_EX_RD2),
    .ID_EX_PC(ID_EX_PC),
    .ID_EX_SgnIMM(ID_EX_SgnIMM)
);
```

```
113
114 ForwardingUnit fu(
115     .ID_EX_MW(ID_EX_MW),
116     .EX_MEM_EnRW(EX_MEM_EnRW),
117     .MEM_WB_EnRW(MEM_WB_EnRW),
118     .EX_MEM_WN(EX_MEM_WN),
119     .MEM_WB_WN(MEM_WB_WN),
120     .ID_EX_RN1(ID_EX_RN1),
121     .ID_EX_RN2(ID_EX_RN2),
122     .StoreHazard(StoreHazard),
123     .ForwardA(ForwardA),
124     .ForwardB(ForwardB)
125 );
126
127 Stalling stalling (
128     .ID_EX_MR(ID_EX_MR),
129     .ID_EX_WN(ID_EX_WN),
130     .IF_ID_RN1(IF_ID_RN1),
131     .IF_ID_RN2(IF_ID_RN2),
132     .ST(ST),
133     .IFIDWrite(IFIDWrite),
134     .PCWrite(PCWrite)
135 );
136
137 MUXALUSrcB muxalusrcb(
138     .ALUSrcB(ID_EX_ALUSrcB),
139     .ID_EX_RD2(ID_EX_RD2),
140     .ID_EX_SgnIMM(ID_EX_SgnIMM),
141     .ALUSelB(ALUSelB)
142 );
143
144 MUXForwardA muxforwarda(
145     .ForwardA(ForwardA),
146     .EX_MEM_ALUResult(EX_MEM_ALUResult),
147     .WD(WD),
148     .ID_EX_RD1(ID_EX_RD1),
149     .ALUInA(ALUInA)
150 );
```

```
152 MUXForwardB muxforwardb(
153     .ForwardB(ForwardB),
154     .EX_MEM_ALUResult(EX_MEM_ALUResult),
155     .WD(WD),
156     .ALUSelB(ALUSelB),
157     .ALUInB(ALUInB)
158 );
159
160 ALUControl alucontrol(
161     .ID_EX_ALUOp(ID_EX_ALUOp),
162     .ALUControlSignal(ALUControlSignal)
163 );
164
165 ALU alu(
166     .ALUInA(ALUInA),
167     .ALUInB(ALUInB),
168     .ALUControlSignal(ALUControlSignal),
169     .ALUResult(ALUResult)
170 );
171
172 EX_MEM exmem(
173     .Clk(Clk),
174     .ID_EX_MReg(ID_EX_MReg),
175     .ID_EX_EnRW(ID_EX_EnRW),
176     .ID_EX_MW(ID_EX_MW),
177     .ID_EX_MR(ID_EX_MR),
178     .ID_EX_WN(ID_EX_WN),
179     .ALUResult(ALUResult),
180     .ID_EX_PC(ID_EX_PC),
181     .ID_EX_RD2(WriteData),
182     .EX_MEM_MReg(EX_MEM_MReg),
183     .EX_MEM_EnRW(EX_MEM_EnRW),
184     .EX_MEM_MW(EX_MEM_MW),
185     .EX_MEM_MR(EX_MEM_MR),
186     .EX_MEM_WN(EX_MEM_WN),
187     .EX_MEM_ALUResult(EX_MEM_ALUResult),
188     .EX_MEM_PC(EX_MEM_PC),
189     .EX_MEM_RD2(EX_MEM_RD2)
190 );
191
```

```
191
192 MUXWriteDataSrc muxwritedatasrc (
193     .StoreHazard(StoreHazard),
194     .ID_EX_RD2(ID_EX_RD2),
195     .EX_MEM_ALUResult(EX_MEM_ALUResult),
196     .WriteData(WriteData)
197 );
198
199 DataMemory dm(
200     .Address(EX_MEM_ALUResult),
201     .WriteData(EX_MEM_RD2),
202     .MemRead(EX_MEM_MR),
203     .MemWrite(EX_MEM_MW),
204     .ReadData(ReadData)
205 );
206
207 MEM_WB memwb(
208     .Clk(Clk),
209     .EX_MEM_MReg(EX_MEM_MReg),
210     .EX_MEM_EnRW(EX_MEM_EnRW),
211     .EX_MEM_WN(EX_MEM_WN),
212     .ReadData(ReadData),
213     .EX_MEM_ALUResult(EX_MEM_ALUResult),
214     .MEM_WB_MReg(MEM_WB_MReg),
215     .MEM_WB_EnRW(MEM_WB_EnRW),
216     .MEM_WB_WN(MEM_WB_WN),
217     .MEM_WB_ALUResult(MEM_WB_ALUResult),
218     .MEM_WB_ReadData(MEM_WB_ReadData)
219 );
220
221 MUXRegWriteSrc muxregwritesrc(
222     .MEM_WB_MReg(MEM_WB_MReg),
223     .MEM_WB_ALUResult(MEM_WB_ALUResult),
224     .MEM_WB_ReadData(MEM_WB_ReadData),
225     .WD(WD)
226 );
227
228 endmodule
```

Testbench: ProcessorPipelineTB

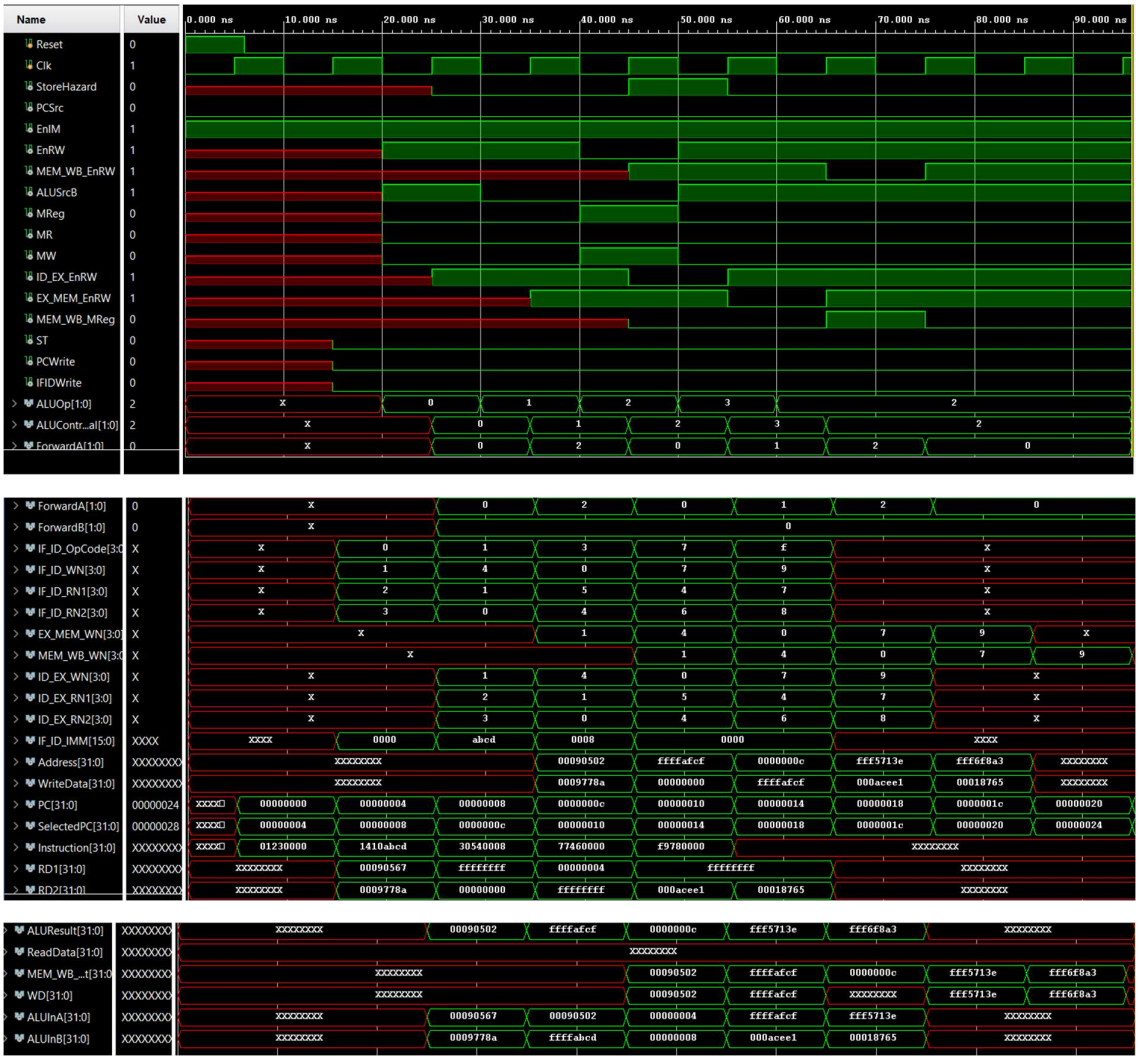
```
1  `timescale 1ns / 1ps
2
3  module ProcessorPipelineTB();
4
5  reg Reset=0;
6  reg Clk=0;
7
8  ProcessorPipeline uut(.Clk(Clk),.Reset(Reset));
9
10 wire StoreHazard,PCSrc, EnIM, EnRW, MEM_WB_EnRW, ALUSrcB, MReg, MR, MW, ID_EX_EnRW, EX_MEM_EnRW, MEM_WB_MReg,ST,PCWrite,IFIDWrite;
11 wire [1:0] ALUOp, ALUControlSignal,ForwardA,ForwardB;
12 wire [3:0] IF_ID_OpCode, IF_ID_WN, IF_ID_RN1, IF_ID_RN2,EX_MEM_WN,MEM_WB_WN,ID_EX_WN,ID_EX_RN1,ID_EX_RN2;
13 wire [15:0] IF_ID_IMM;
14 wire [31:0] Address, WriteData, PC, SelectedPC, Instruction, RD1, RD2, ALUResult, ReadData, MEM_WB_ALUResult, WD,ALUInA,ALUInB;
15
16
17 assign PC = uut.pc.PC;
18 assign SelectedPC = uut.muxpcsrc.SelectedPC;
19
20
21 assign Instruction = uut.im.RD;
22 assign IF_ID_RN1 = uut.ifid.IF_ID_RN1;
23 assign IF_ID_RN2 = uut.ifid.IF_ID_RN2;
24 assign IF_ID_IMM = uut.ifid.IF_ID_IMM;
25 assign ID_EX_RN1 = uut.idex.ID_EX_RN1;
26 assign ID_EX_RN2 = uut.idex.ID_EX_RN2;
27 assign ID_EX_WN = uut.idex.ID_EX_WN;
28 assign EX_MEM_WN = uut.exmem.EX_MEM_WN;
29 assign MEM_WB_WN = uut.memwb.MEM_WB_WN;
30 assign ALUInB = uut.alu.ALUInB;
31 assign ALUInA = uut.alu.ALUInA;
32 assign ALUResult = uut.alu.ALUResult;
33 assign RD1 = uut.regfile.RD1;
34 assign RD2 = uut.regfile.RD2;
35
36 assign IF_ID_OpCode = uut.ifid.IF_ID_OpCode;
37 assign IF_ID_WN = uut.ifid.IF_ID_WN;
```

```
assign IFIDWrite = uut.stalling.IFIDWrite;
assign PCWrite = uut.stalling.PCWrite;
assign ST = uut.stalling.ST;
assign PCSrc = uut.mc.PCSrc;
assign EnIM = uut.mc.EnIM;
assign EnRW  = uut.mc.EnRW;
assign ALUSrcB = uut.mc.ALUSrcB;
assign MReg = uut.mc.MReg;
assign MR = uut.mc.MR;
assign MW = uut.mc.MW;
assign ALUOp = uut.mc.ALUOp;
assign ForwardA = uut.fu.ForwardA;
assign ForwardB = uut.fu.ForwardB;
assign StoreHazard = uut.fu.StoreHazard;
assign ID_EX_EnRW = uut.idex.ID_EX_EnRW;
assign EX_MEM_EnRW = uut.exmem.EX_MEM_EnRW;

assign ALUControlSignal = uut.alucontrol.ALUControlSignal;

assign Address = uut.dm.Address;
assign WriteData = uut.dm.WriteData;
assign ReadData = uut.dm.ReadData;

assign MEM_WB_ALUResult = uut.memwb.MEM_WB_ALUResult;
assign MEM_WB_MReg = uut.memwb.MEM_WB_MReg;
assign MEM_WB_EnRW = uut.memwb.MEM_WB_EnRW;

assign WD = uut.muxregwritesrc.WD;

always #5 Clk= ~Clk;
```

```
initial
begin
$monitor("Vtime=%t value in Reg1=%h,Reg4=%h,Reg7=%h,Reg9=%h, at memlocation 12=%h,13=%h,14=%h,15=%h",
        $time,uut.regfile.Registers[1],uut.regfile.Registers[4],uut.regfile.Registers[7],
        uut.regfile.Registers[9],uut.dm.DataMem[12],uut.dm.DataMem[13],uut.dm.DataMem[14],uut.dm.DataMem[15]);
Clk=0;
uut.im.InstMem[0]=8'b0000_0001;
uut.im.InstMem[1]=8'b0010_0011;
uut.im.InstMem[2]=8'b00000000;
uut.im.InstMem[3]=8'b00000000;

uut.im.InstMem[4]=8'b0001_0100;
uut.im.InstMem[5]=8'b0001_0000;
uut.im.InstMem[6]=8'b1010_1011;
uut.im.InstMem[7]=8'b1100_1101;

uut.im.InstMem[8]=8'b0011_0000;
uut.im.InstMem[9]=8'b0101_0100;
uut.im.InstMem[10]=8'b0000_0000;
uut.im.InstMem[11]=8'b0000_1000;

uut.im.InstMem[12]=8'b0111_0111;
uut.im.InstMem[13]=8'b0100_0110;
uut.im.InstMem[14]=8'b0000_0000;
uut.im.InstMem[15]=8'b0000_0000;

uut.im.InstMem[16]=8'b1111_1001;
uut.im.InstMem[17]=8'b0111_1000;
uut.im.InstMem[18]=8'b0000_0000;
uut.im.InstMem[19]=8'b0000_0000;

uut.regfile.Registers[0]=32'h00000000;
uut.regfile.Registers[1]=32'hFFFFFFFF;
uut.regfile.Registers[2]=32'h00090567;
uut.regfile.Registers[3]=32'h0009778A;
uut.regfile.Registers[4]=32'hFFFFFFFF;
uut.regfile.Registers[5]=32'h00000004;
uut.regfile.Registers[6]=32'h000ACEE1;
uut.regfile.Registers[7]=32'hFFFFFFFF;
uut.regfile.Registers[8]=32'h00018765;
```

```
110 
111   Reset=1;#6;Reset=0;
112   #90;
113   $finish;
114 end
115 
116 endmodule
```

Simulation results:



Register values in TCL console:

```
# run 1000ns
Vtime=                  0 value in Reg1=ffffffff,Reg4=ffffffff,Reg7=ffffffff,Reg9=xxxxxxxx, at memlocation 12=xx,13=xx,14=xx,15=xx
Vtime=              50000 value in Reg1=00090502,Reg4=ffffffff,Reg7=ffffffff,Reg9=xxxxxxxx, at memlocation 12=xx,13=xx,14=xx,15=xx
Vtime=              55000 value in Reg1=00090502,Reg4=ffffffff,Reg7=ffffffff,Reg9=xxxxxxxx, at memlocation 12=ff,13=ff,14=af,15=cf
Vtime=              60000 value in Reg1=00090502,Reg4=ffffafcf,Reg7=ffffffff,Reg9=xxxxxxxx, at memlocation 12=ff,13=ff,14=af,15=cf
Vtime=              80000 value in Reg1=00090502,Reg4=ffffafcf,Reg7=fff5713e,Reg9=xxxxxxxx, at memlocation 12=ff,13=ff,14=af,15=cf
Vtime=              90000 value in Reg1=00090502,Reg4=ffffafcf,Reg7=fff5713e,Reg9=fff6f8a3, at memlocation 12=ff,13=ff,14=af,15=cf
```

Description of the blocks:

1. **Program Counter (PC)**:
    - Holds the address of the next instruction to execute.
    - Updates with each clock cycle or based on branch/jump decisions.
2. **Instruction Memory**:
    - Stores the program's instructions.
    - Outputs the current instruction based on the PC's address.
3. **Registers (Register File)**:
    - Contains multiple registers storing operands/data.
    - Provides source operands (RD1, RD2) for instructions and accepts results (WData).
4. **Hazard Detection Unit**:
    - Detects pipeline hazards (data, structural).
    - Controls stalls or delays in pipeline to resolve these hazards.
5. **Control Unit**:
    - Generates control signals required for operation (ALU operations, memory access, register write signals).
    - Coordinates operations across pipeline stages.
6. **ALU Control**:
    - Determines the exact operation that the Arithmetic Logic Unit (ALU) should perform.
    - Based on instruction type and opcode received from the control unit.
7. **Arithmetic Logic Unit (ALU)**:
    - Performs arithmetic and logical operations.
    - Outputs computation results for storage or further use in pipeline.
8. **Data Memory**:
    - Provides storage for data used during execution.
    - Enables reading and writing data for load/store instructions.
9. **Forwarding Unit**:
    - Detects data hazards where subsequent instructions require results that are not yet written back.

- Provides forwarding paths to bypass results directly to earlier stages, avoiding stalls.

10. **Pipeline Registers (IF/ID, ID/EX, EX/MEM, MEM/WB)**:
    - Temporarily store intermediate values between pipeline stages.
    - Pass instruction data and control signals from one stage to the next, ensuring synchronization across pipeline operations.
11. **Sign Extend**:
    - Extends immediate values from instructions to the processor's full word length.
    - Ensures immediate values can be correctly operated on by the ALU.